Composing Cognitive Theories in CogLean: Multi-Digit Addition

Jeff Shrager

Draft, November 2026

Draft for Comment[1]

Abstract

CogLean is a library of cognitive theory in the Lean proof assistant, organized on the model of Mathlib: mechanisms are defined once, phenomena are stated independently of any mechanism, and theorems relating them are machine-checked and reusable. A companion paper introduces the library and its account of theorems as explanations. This paper asks whether independently formalized theories, composed in such a library, yield meaningful consequences that would otherwise require building and investigating a new integrated model. Two independently formalized theories, EPAM's discrimination-net recognition (Feigenbaum and Simon) and the distributions-of-associations model of strategy choice (Siegler and Shrager), are composed with a general problem solver into an executable model of three-digit addition, and their established theorems are reused to derive consequences of the composition. The results form a hierarchy. At the most general level, a theorem relates recognition, associative retrieval and backup use for any problems and answers: a problem confused with a familiar one exhibits a theory-neutral *confusion-error pattern*. In the integrated model, that theorem is a deduction about the same executable trial that produces the model's answers. The pattern is also proved for *every* memory that satisfies three stated laws; EPAM's memory has all three, and ACT-R's declarative memory, proved for every parameter setting, has none. Further results are compositional: whole-problem errors do not depend on the problem solver, wrong answers have two sources with different signatures, and one memory can serve both whole problems and column facts. Other results separate what follows from a mechanism regardless of representation from what requires particular representational properties. Concrete settings, with exact values relegated to an appendix, serve as witnesses that the hypotheses can be met and that model families diverge, and an independent blind simulation checks the formal model against its prose specification. The multi-digit model is a demonstration of method, not a theory of arithmetic.

1 Introduction

CogLean [1] is a library of cognitive theory in the Lean proof assistant, organized on the model of Mathlib, the community library of formalized mathematics [2, 3]. The companion paper [4] introduces it: its organization, its vocabulary of mechanisms, phenomena, theorems and conditions, and its central claim, that CogLean's theorems are explanations, deductions of phenomena from mechanisms and stated assumptions, proved about the same definitions that run as the model. It illustrates the approach with small examples. This paper is the first extended test. It asks a question cognitive architectures have long addressed by building integrated models [5–7]:

[1] This draft has *not* been carefully checked by the author. It is likely to contain erroneous citations and details. Think of this as an over-starched blog post. I'm happy to receive constructive feedback at: `jshrager@gmail.com`

If independently developed cognitive theories are represented in a common formal library, can we derive scientifically meaningful consequences from their composition, consequences that would otherwise require building and investigating a new integrated model from scratch?

We give a concrete, checked answer in one case. The case is three-digit addition, composed from EPAM's discrimination nets [8, 9], Siegler and Shrager's distributions-of-associations (DOA) model of strategy choice [10], and a problem solver in the tradition of GPS and repair theory [11, 12]. Arithmetic is the demonstration, not the objective. We are not proposing a new theory of children's arithmetic. Multi-digit addition is a familiar task, complex enough to need several mechanisms, and two of the needed theories already exist.

The composition tests three things. That the mechanisms compose: the integrated model runs, with no answer supplied to it. That the *theorems* compose: results proved for each component transfer to the whole, so that the integrated model's consequences are derived largely by reusing them. And that the derived explanations stay aligned with the program: the trial that is executed to produce the model's answers is the same trial the theorems reason about, through recognition, retrieval and problem solving, so a change to any mechanism re-checks every explanation that depends on it.

The numerical predictions of the example are illustrative. They depend on chosen problems, learning histories, encodings, strengths, criteria and backup procedures, and a simulation could produce comparable numbers. Their roles are to show that the composition runs, that the theorems' hypotheses can be met, and that model families can diverge [4]. They are collected in Appendix B.

2 Background: CogLean

CogLean has four layers (Figure 1): Lean's core, with every probability an exact rational number; CogLib, the general library of representations, memories, processes, an interface any memory can implement, and theory-neutral phenomena and laws, which imports no theory; theories, each stating its postulates as choices of CogLib constructs, cited to their sources; and applications, models of particular tasks assembled from theories and library mechanisms.

The vocabulary is that of the companion paper, whose Table 1 maps it onto scientific notions. *Mechanisms* are definitions, chosen by a theory. *Representations* are parameters of the mechanisms, with their properties stated separately. *Phenomena* are predicates over a task's outcomes, stated without any mechanism: targets, which assert nothing until a mechanism is proved to satisfy them; observed, they are facts, and derived but unobserved, predictions. *Theorems*, with their chains of lemmas, are explanations: a mechanism, or every mechanism of a class, produces behavior satisfying a predicate. *Conditions* are their hypotheses: situational ("problem x is confused with problem q"), empirical premises that remain unexplained ("q's associations are sharp"), or well-formedness requirements ("the strengths have positive total"). *Laws* are properties of mechanisms, proved for a family or required by a theorem about a class; none is a Lean `axiom`. A theorem that has a phenomenon among its hypotheses is a relation between phenomena, not a prediction of a model. A value computed at one setting is a witness, not an explanation.

Shared definitions make theories comparable at the level of proofs. For example, CogLib proves once that any learner whose learning never disturbs what it already knows reaches a learning criterion (`criterion_of_nonInterference`). It separately proves that discrimination-net learning never disturbs prior learning (`netLearner_nonInterference`). EPAM's learning to criterion then follows in a line, and so will that of any later theory whose learning is non-interfering.

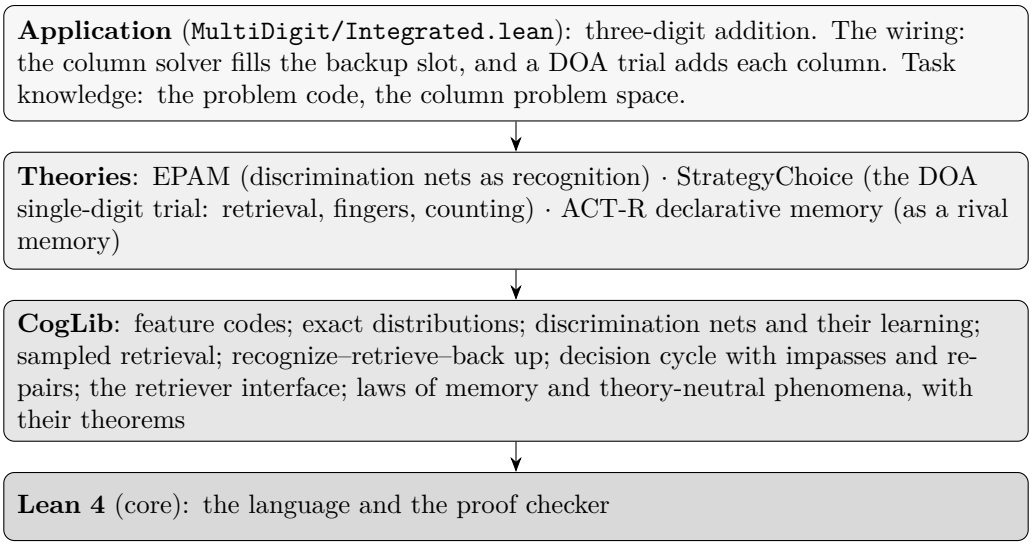


Figure 1: The layers of CogLean. Arrows point from a layer to what it imports. Each layer's theorems are available to the layers above it.

3 Components, formalized independently

3.1 EPAM: recognition by discrimination

EPAM [8, 9] recognizes a stimulus by sorting its code, a vector of features, down a discrimination net of tests to a stored image. When a stimulus is confused with a stored one, learning grows a test at that point, on the first feature in a noticing order where the two differ. We formalize the propositions of Feigenbaum's report, not the original IPL-V program. CogLib proves, for every net, code and noticing order:

- recognition depends only on the features tested on the path a code follows;
- learning a stimulus never changes the recognition of another (`learn_preserves`);
- a confusion agrees with the stored image on every feature tested on its path (`confusion_agrees_on_path`);
- every net learned from an empty net stores each image once and recognizes each as itself (`learnAll_clean`).

3.2 DOA: distributions of associations

In the DOA model [10], a child adding single digits has associations of varying strength between each problem and possible answers.

1. On each trial the child has a confidence criterion c and a search length n.
2. A search samples an answer in proportion to strength, and states it if its strength exceeds c, for up to n searches.
3. If retrieval is exhausted, the child puts up fingers, which adds strength to the answer that many fingers represents, and retrieves once more.

4. Failing that, the child counts the fingers, with fixed chances of skipping or double-counting an object.

5. Every stated answer is strengthened, a correct one more than an incorrect one.

Let R be the chance that one search clears the criterion. StrategyChoice proves for every distribution, criterion and search length:

- retrieval is used with probability $1 - (1 - R)^n$ (`SC.retrieval_prob`);
- the error rate among retrieval trials is the wrong answers' share of the strength above the criterion, whatever the search length (`SC.retrieval_error_share`);
- neither depends on the backup strategies (`SC.retrieval_independent_of_backup`);
- the error rate among counting trials does not depend on the associations at all, which is the chapter's "nonintuitive prediction" (`SC.counting_error_share`).

3.3 A discovery from formalization alone

The DOA chapter's central idea is that the peakedness of a problem's distribution of associations governs both accuracy and strategy use. It states that "a peaked distribution results in less frequent use of overt strategies" [10, p. 245]. Formalizing the model showed that half of this follows from the postulates and half does not.

- **What follows.** Moving strength to the correct answer never raises the error rate on retrieval trials, at any criterion or search length (`SC.peaking_lowers_retrieval_errors`), and neither does learning from a correct answer (`SC.correct_strengthening_lowers_errors`). These are general theorems.
- **What does not, for arbitrary redistributions.** Peaking can *raise* overt strategy use. Take $1 + 1$ with answers 2, 3 and 1, and strengths $.38, .31, .31$. Move one hundredth of strength from the wrong answer 3 to the correct answer 2, which is already the strongest: $.39, .30, .31$. With search length 1, averaged over the chapter's own twelve confidence criteria, predicted overt strategy use rises from 7/12 to 73/120 (`SC.peaking_can_raise_overt_use`). The reason is that the weakened wrong answer falls to the criterion .30, so it no longer clears it. Searches that used to state it now fail, and the child turns to fingers. Note that this change *moves* strength from a wrong answer to the correct one. It is a change to the distribution, not one produced by DOA's learning rule, which *adds* strength to the stated answer without taking any from its competitors.
- **What follows for learning.** Under DOA's additive rule, learning from a correct answer never raises backup use when that answer exceeds the criterion (`SC.correct_strengthening_never_raises_backup`), and in particular when it is already the strongest (`SC.dominant_strengthening_never_raises_backup`). Under an alternative rule that renormalizes strengths after learning, the same learning event can raise backup use (`SC.normalized_strengthening_can_raise_backup`; worked through in the companion paper, Appendix A).

The counterexample therefore disproves an unrestricted claim about peaking, but it does not show that the original DOA learning mechanism produces increased backup use. Read as a claim

about arbitrary changes to a distribution, the chapter's statement needs a condition it does not state, roughly that no competitor sits near a criterion. Read as a claim about learning, it holds under the additive rule and fails under renormalization, an assumption the chapter does not settle. Nothing about this required composition or new mechanisms. It came from stating the postulates precisely and asking whether the informal claim followed. Formalization did not so much correct the theory as expose an ambiguity in it and establish which conclusions follow from which assumptions.

3.4 General problem solving

CogLib's problem-solving framework (`CogLib/Process/ProblemSolving.lean`) is task-general:

- a *problem space* has states and operators; each operator has a precondition and an effect, and an effect is a distribution over states, so an operator can include a probabilistic process;
- a *decision cycle* runs a procedure that proposes an operator in each state; a proposed operator whose precondition is violated is an *impasse*, which repair heuristics, filtered by critics, resolve [12].

CogLib also implements GPS-style means–ends analysis, with goals, subgoals and backtracking [11]. Neither knows anything about arithmetic.

4 Composing theories: multi-digit addition

4.1 The composition

A child adding two three-digit numbers may know the answer to a familiar problem such as 100+100, or may work it out column by column. The model nests DOA's architecture one level up (Figure 2). It has six parts.

1. **A problem code**: features for each digit of each addend. Agreeing on a digit's features means having the same digit (postulate M1, `MultiDigit/Postulates.lean`).
2. **Recognition** by an EPAM net that stores familiar problems (M2).
3. **Whole-problem retrieval**: DOA sampling from the recognized problem's associations (M3).
4. **A column solver**, invoked when nothing is recognized or retrieval is exhausted (M4). It is a problem space with operators to add a column, write a digit, write and carry, and finish, run by a procedure (`MultiDigit/Solver.lean`).
5. **A DOA single-digit trial** for each column's addition, with retrieval, fingers and counting (`MD.doaFact`, wrapping `SC.trial`).
6. **CogLib's decision cycle**, which sequences the operators and handles carrying, impasses and repairs (`CogLib/Process/ProblemSolving.lean`).

In Lean, the composition is three definitions (`MultiDigit/Integrated.lean`):

```
def solverBackup (F : Facts) (p : Problem) : Dist Nat :=
  Solver.answers F.fact Solver.full Solver.lenient p
def itrial E N str as cc n F x := trial E N str as cc n (solverBackup F) x
```

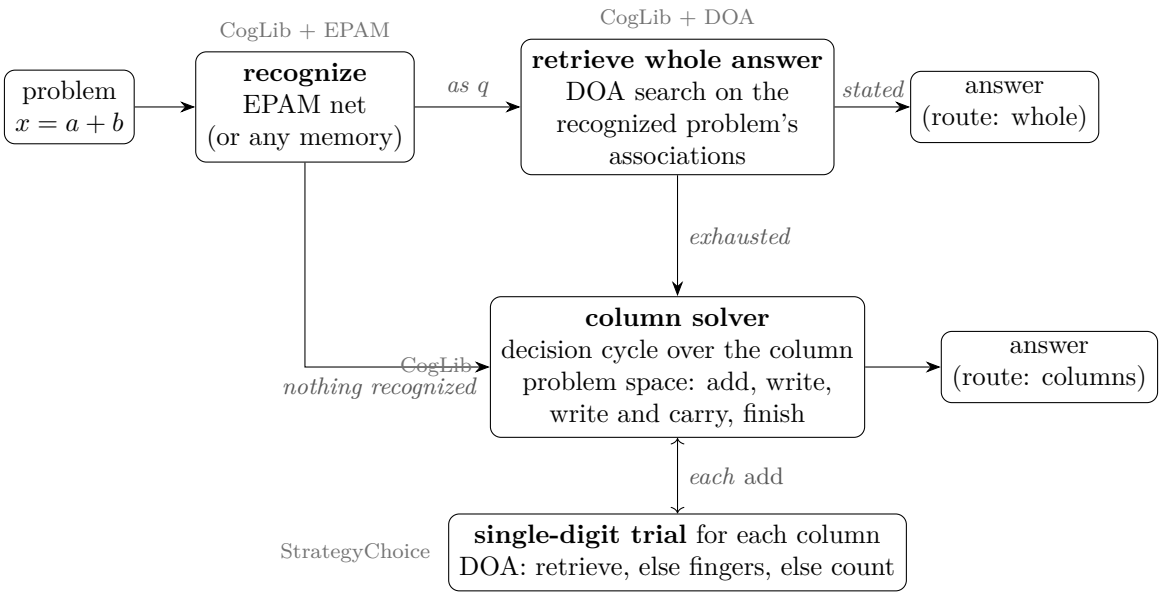


Figure 2: One trial of the integrated model (`MD.Integrated.itrial`). Grey labels give the source of each mechanism. When x is recognized as a familiar q, the whole answer is retrieved from q's associations. The solver runs only when recognition fails or retrieval is exhausted, and it computes the answer from single-digit trials and a carry rule.

Here `F.fact` is the single-digit DOA trial, `Solver.full` the full column procedure, and `trial` the recognize-and-retrieve trial from `MultiDigit/Postulates.lean`. The integrated model introduces no new mechanism. Its only correct arithmetic knowledge is in the single-digit associations, which are peaked on each digit pair's sum, as in DOA, and in the carry rule.

4.2 Why the integrated model is stronger evidence

An earlier version of the example, still in the repository (`MultiDigit/Example.lean`), checked the general theorems with a backup that simply returned the correct sum, `Dist.pure p.sum`. That suffices to instantiate theorems that hold for *every* backup. But it supplies the answer instead of computing it, so it is not evidence that the mechanisms compose. In the integrated model, nothing is handed the answer. The answer 250 to 150+100 is assembled by the solver from three DOA trials and a carry rule, and it is never certain. So the composition must work at the level of mechanisms (the parts fit and run together) and at the level of theorems (the parts' results transfer to the whole).

4.3 The executable backup: arithmetic as general problem solving

When whole-problem retrieval fails, the integrated model invokes an executable column-addition solver. It is an instance of the general framework of Section 3.4, not a separate arithmetic program. The arithmetic instance (`MultiDigit/Solver.lean`) defines operators to add a column, write its digit, write and carry, and finish. The procedure `full` proposes them as the calculation progresses, units first. `Solver.answers` runs CogLib's decision cycle, for a bounded number of cycles (eight, enough for three columns), and reads off the written answer. In `solverBackup`, `F.fact` supplies the single-digit process and `Solver.lenient` the repair policy; with the full procedure, no impasse arises.

Each *add* operator performs a DOA trial on its column's digits and returns a distribution of possible digit sums. The framework composes such steps with `Dist.bind`, which carries every possible outcome, with its probability, through the operations that follow. Nothing is sampled, and nothing consults the correct answer. The result is a distribution over complete multi-digit answers, errors included.

Means–ends analysis has been instantiated for the same problem space (`MultiDigit/SolverGPS.lean`), where it is proved to work units first and to answer every problem exactly when the single-digit facts are exact. The integrated model uses the decision cycle; the GPS results concern goal-directed operator selection.

4.4 Composition of theorems

Three kinds of reuse make the composition deductive rather than merely executable.

A bridge. There are two executable descriptions of column addition. `columns` (`MultiDigit/Columns.lean`) is a direct implementation of probabilistic column arithmetic. `answers` performs the same task on the decision cycle just described. The bridge theorem proves that they produce identical answer distributions, for every problem and every single-digit process:

```
theorem bridge (fact : Nat → Nat → Dist Nat) (p : Problem) :
    answers fact full lenient p = columns fact p
```

The equivalence is proved, not established by testing examples. Every theorem proved about `columns` therefore holds for the solver; for instance, that a column error is a single-digit error moved to its place when the other columns are exact (`MD.column_error_is_doa_error`). The bridge equates output distributions, not internal processes. As accounts of what a child does, the two descriptions still differ.

Associativity. Each arrow in Figure 2 is a two-stage draw of finite distributions. CogLib proves that such draws regroup (`Dist.bind_assoc`), so a theorem about a component holds where the component sits inside the whole.

Well-formedness, inherited. The general confident-errors theorem needs the backup to have total probability 1. For the integrated model this is proved, not assumed, by chaining results from three packages (`mass_solverBackup`):

- DOA retrieval is proper (CogLib, `mass_search`);
- a whole DOA trial is proper (StrategyChoice, `SC.mass_trial`);
- the column procedure is proper when its facts are (MultiDigit, `mass_columns`);
- the solver equals the column procedure (`Solver.bridge`).

A parallel chain proves that every weight is nonnegative, given nonnegative strengths and rates (`nonneg_search`, `SC.nonneg_trial`, `nonneg_solverBackup`, `nonneg_itrial`). Both are needed: CogLib's distributions are lists of rational weights, and total mass 1 alone does not exclude negative weights. These lemmas contribute no psychology. They are what allows theorems written for an abstract backup to apply to a concrete, computing one, and the integrated trial to be read as a probability distribution.

5 A theory-neutral phenomenon

In DOA alone, a problem's errors and its use of backup strategies come from the same distribution. A flat distribution of associations produces both more retrieval errors and more backup use, so errors and hesitation go together. Recognition can break this tie: a problem confused with a familiar one is retrieved from the familiar problem's distribution, with that problem's backup use and with its answers, which are wrong for the confused problem.

CogLib states the resulting pattern once, as a predicate over answer and strategy-route distributions (`CogLib/Phenomena/ConfusionErrors.lean`):

```
def ConfusionErrorPattern trial wholeWrong isWhole isBackup x q : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧                              -- (i)
  prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧         -- (ii)
  prob (trial x) isBackup = prob (trial q) isBackup                -- (iii)
```

For any problems and outcomes: (i) the familiar q is never answered wrongly by retrieval; (ii) every retrieved answer to x is wrong; (iii) x uses the backup exactly as often as q. The definition names no mechanism, and none of the conditions under which mechanisms produce the pattern is part of it. They belong to the theorems. Mechanism-independent is not the same as directly observable, however: (ii) and (iii) refer to the route that produced an answer, whole-problem retrieval or the backup, which an answer alone does not reveal. Applying the predicate to people requires identifying or measuring the route, for instance by latency, verbal report or observed overt work, as the DOA studies classified strategies.

Equal backup use is not low backup use. CogLib therefore states a second, stronger phenomenon, `ConfidentErrors r`: the pattern with backup use at most r. The bound is an argument, not a fixed threshold, so the predicate does not build in an arbitrary standard of "confidence". Two predicates for one informal idea is the point: the informal notion of a confident error turns out to combine a structural relation, which holds whenever recognition hands one problem another's associations, and a quantitative one, which depends on how well those associations are learned.

6 General deductions

The results of this section hold for every encoding, net, strength set, criterion, search length and single-digit process, under the conditions each states. None depends on the illustrative numbers.

6.1 Confident errors

The theorem, as stated for the integrated model (lightly abridged):

```
theorem integrated_confident_errors (E : Encoding) (N : Net) (str ...) (F : Facts)
    (hF : F.Proper) (x q : Problem) (hT : ∀ y, total as (str y) ≠ 0)
    (hq : N.select (E.code q) = some (E.code q))      -- q is recognized as itself
    (hx : N.select (E.code x) = some (E.code q))      -- x is confused with q
    (hs : Sharp str cc (E.code q) q)                  -- only q's sum clears the criterion
    (hne : x.sum ≠ q.sum) :
    ConfusionErrorPattern (itrial ...) wholeWrong isWhole isColumns x q
```

The confused problem is answered wrongly with the same fluency as the familiar one. Whether that fluency is high is separate: since a net sorts x to q for certain, the rate of backup use on x is $(1 - R_q)^n$, where R_q is the chance that one search of q's associations clears the criterion (`integrated_backup_rate`), and if $R_q \geq r$ the model has `ConfidentErrors` with bound $(1 - r)^n$

(`integrated_confident_errors_le`). With flat associations on q the pattern still holds and the errors are not confident.

What is assumed. The postulates are the mechanisms: the net, DOA retrieval and the solver. The conditions are of three kinds.

- *Recognition conditions* (`hq`, `hx`): q is recognized as itself and x is confused with it. These are properties of a particular net. They are not arbitrary: every net EPAM learns recognizes its images as themselves (`learnAll_clean`), and the net in our setting, learned by EPAM from $100+100$ and $100+200$, is verified to confuse $150+100$ with $100+100$ (`Example.net_facts`).
- *Sharpness* (`hs`): only q's correct sum clears the criterion. This is an empirical premise that the library does not yet derive from learning.
- *Well-formedness* (`hF`, `hT`): strengths have positive totals.

No hypothesis mentions confident errors, or any relation between errors and backup use.

What is reused. The proof is one line. It applies `MD.confident_errors`, the theorem for an arbitrary backup, with `mass_solverBackup` for its well-formedness hypotheses. `MD.confident_errors` is in turn a relabelling of CogLib's `RR.confident_errors`, which holds for any task with problems, a familiar set and a backup. That theorem combines three earlier lemmas:

- a problem recognized as a sharp q gets only q's answer (`confusion_gives_neighbours_answer`);
- so all its whole-problem answers are wrong when the sums differ (`wrong_eq_whole_of_sharp`);
- backup use after certain recognition as y is $(1 - R_y)^n$, which depends only on y (`backup_prob_certain`).

The last lemma is DOA's retrieval-probability result, lifted to a recognized item.

6.2 Whole-problem errors do not depend on the solver

Once x is recognized as q, the chance of a whole-problem answer with any property is that of retrieval on q's associations, whatever the backup (`whole_independent_of_solver`). This is a general consequence of the composition, at every setting. Its psychological content is a prediction about intervention: improving column skill, for instance by teaching carrying, cannot reduce errors that arise from mistaken recognition. Only learning that changes recognition or associations can.

6.3 Two sources of error

Under the same conditions as confident errors, the integrated model distinguishes two sources of error on a confused problem (`errors_two_sources`).

- **Whole-route errors** are always the confused neighbour's answer (here 200), whenever that route is taken.
- **Column-route errors** occur with probability equal to the chance that retrieval is exhausted times the solver's own error rate on x, which comes entirely from the single-digit DOA trials.

This result is *compositional*. Neither recognition nor DOA says it alone, and it would not hold with the old backup, which never erred.

The two sources tend to leave different signatures, but only the first is guaranteed. A whole-route error is always the confused neighbour's sum. A column-route error has a simple signature only when one column errs: if the other columns are exact, the problem is wrong exactly when that column's DOA trial is, and off by $10^j\delta$ when column j errs by δ (`solver_error_is_doa_error`). Several columns can err on the same trial, and a column error can propagate through a carry. Errors can even cancel. Exact single-digit facts in every column are sufficient for an exact answer (`solver_exact_of_exact`), but not necessary: for $150 + 100$, a units result of 10 and a tens result of 4 still give 250 (`compensating_errors`).

6.4 One memory for whole problems and column facts

In the integrated model, the solver's single-digit facts come from a DOA association table of their own, separate from the memory that recognizes whole problems. A child has one memory. `MultiDigit/SharedMemory.lean` removes the separation, again by composition alone. Each column's digit pair $d + e$ is coded in the problems' own code space, as the problem $d + e$. A column's addition is then recognize–retrieve–back up on that pair (`memFact`): the *same* retriever that recognizes whole problems calls up a stored code, and retrieval searches its associations in the same strength table. The memory becomes a single parameter, used at both levels; the solver is unchanged, since it is parametric in its facts.

Three results follow from theorems already in the library:

- **The separate table is a special case.** If the memory recognizes a digit pair as itself, the column fact *is* the DOA fact on that pair's associations (`memFact_self`). So when each column's pair is recognized as itself, the shared model is the integrated model (`strial_reduces`, `strial_net_reduces`), and every theorem above holds of it.
- **Confident errors one level down.** A digit pair taken for another, whose associations are sharp, exhibits the confusion-error pattern (`column_confident_errors`). The proof applies the general theorem to digit pairs; it needed no new lemma.
- **Noninterference needs a condition.** EPAM's practiced problems are never disturbed by learning, by a theorem that holds the backup fixed (`MD.learning_preserves_trial`). With one memory, learning can change the backup itself: a digit pair the net has not learned may be sorted differently after it learns something new. The result transfers when the problem and its three digit pairs were all learned (`shared_learning_preserves_trial`). Without that condition, noninterference for the shared model is not proved.

7 Classes of mechanisms

The theorems so far quantify over representations and parameters, but each concerns one kind of memory. The next step is to quantify over memories.

A result for every memory satisfying laws. `RR.confident_errors_of_laws` proves the confusion-error pattern for *any* memory, given only three laws, stated theory-neutrally in CogLib (`Phenomena/RetrievalLaws.lean`) against a common retriever interface:

- *determinacy*: retrieval calls up a single item for certain;

- *certainty*: a studied item retrieves itself;
- *stability*: studying one item does not change another's retrieval of itself.

Together with the remaining conditions (a stranger can be taken for a studied item, whose associations are sharp, and the correct answers differ), any memory with the three laws exhibits the pattern. A new theory of memory inherits the result by proving the three laws, without reproving anything about recognition, retrieval or backup. This is a result about a class of models, not about one model. It says that the laws are *sufficient*; it does not say they are necessary, nor that human memory satisfies them.

Which memories are in the class. CogLean's theory matrix (`CogMatrix/`) proves, for each memory family and each law, whether the family has the law at every setting, never has it, or lacks it at some setting. For the two memories here, both cells are proved for every parameter setting:

- EPAM's discrimination net has determinacy, certainty and stability (`Matrix.Discrimination.determinate`, `.certainty`, `.stability`);
- ACT-R's declarative memory [6], with base-level activation, partial matching and a retrieval threshold, has none of them (`Matrix.Activation.never_determinate`, `never_certainty`, `never_stability`).

So the class-level theorem covers EPAM and never covers ACT-R, and the explanation of the difference is a property of memory, not an observation at one setting.

What failing a law does and does not show. That the theorem does not apply to ACT-R does not show that ACT-R cannot produce similar behavior, and at the illustrative setting it does produce graded errors of a similar kind. What can be shown is that the exact pattern fails there: with ACT-R's memory the trial does not satisfy `ConfusionErrorPattern` (`setting_actr_not_pattern`), because the confused problem's backup use differs from the familiar problem's. That is a counterexample at one setting, not a theorem about every setting. A graded analogue of the pattern, with its own laws, would be needed to say what memories like ACT-R's do in general.

8 The role of representation

The specific prediction that $150+100$ is confused with $100+100$ depends on the illustrative binary-coded representation, the order in which features are noticed, and the learning history that built the net. The theorems about what such a confusion entails do not. CogLean makes this separation explicit, and the library already distinguishes three kinds of result.

1. **Representation-independent.** The confusion-error pattern, solver independence and the two sources of error hold for every encoding. The representation enters only through the recognition conditions (which problem is confused with which), and those are hypotheses, not built in.
2. **Requiring representational properties.** Some consequences need the encoding to preserve particular information, stated as properties of encodings, not as particular encodings (`MultiDigit/Features.lean`, `FeaturePhenomena.lean`):

- *Zeros*: each digit has a feature saying whether it is zero. Then a confusion keeps the zeros the net tests (`confusion_keeps_zeros`), and a problem confused with a round problem, by a net that tests the relevant zeros, is answered exactly (`round_confusion_exact`).
- *Magnitude*: each digit has threshold features ("greater than k"). Then a confused digit lies between the same tested thresholds as the true one, and its error is bounded by the gap between them (`confusion_within_gap`).
- *Column facts*: each column has features identifying its single-digit fact. Then a confusion with every column's fact tested costs nothing (`facts_same_sum`), and with only the low columns' facts tested the error is a multiple of 100 or of 10 (`facts_mod_100`, `facts_mod_10`).

Each holds for every encoding with the property, every net, and every memory that agrees on the features concerned; versions with no hypothesis on the net hold for every net EPAM learns from an empty one (`round_confusion_exact_learned`, `facts_same_sum_learned`).

3. **Instance-specific.** Which problems are actually confused, and the resulting probabilities, depend on a particular combination of encoding, noticing order, learning history and strengths. These are the values of Appendix B.

The simulation of Section 11 adds a caution about the middle category: in the learning histories it tested, nets rarely tested enough features for these hypotheses to hold. A theorem that requires a representational property says what follows when the property is used; whether learning makes use of it is a separate question.

9 Concrete realizations

The general results are illustrated at one chosen setting: an encoding, a learned net, associations, criterion and search length (Appendix B). The concrete cases do five things. They show that independently formalized mechanisms compose into a model that runs; that the formalizations have nontrivial realizations, since a learned net does meet the conditions of the confident-errors theorem; that different memory families make different predictions, as constructive witnesses; that the abstract results can be understood through cases; and that the formal model can be checked against an independent implementation (Section 11). Why such values are witnesses rather than explanations is discussed in the companion paper [4]. Table 1 summarizes the comparison qualitatively; exact values are in Tables 3 and 4.

10 Kinds of results

Table 2 sorts the results by what they rest on. A reader should be able to tell which statements are library facts about mechanisms, which are deductions from the composition, which hold only given further conditions, which hold for whole classes of mechanisms, and which are values at chosen settings. Not every theorem is a psychological prediction, and none here is an empirical finding: the conditional, compositional and class-level results become empirical predictions about people only once their conditions can be identified in people and the predictions tested.

	EPAM net	ACT-R memory
Confusion of $150 + 100$ with $100 + 100$	certain	graded; can also retrieve $100 + 200$, or fail
Errors on the confused problem	confident: the neighbour's sum, with the familiar problem's low backup use	similar errors, but backup use differs from the familiar problem's
Practiced problem after new study	undisturbed (every setting, fixed backup)	interference
Column facts from the same memory	no change (pairs recognized as themselves)	digit pairs confused with each other; solver accuracy falls
General status	in the class of the confident-errors theorem	outside it, at every setting

Table 1: The two memories in the same application, qualitatively. The last row is general, and so is EPAM's entry on the practiced problem, which holds at every setting when the backup is fixed (`MD.learning_preserves_trial`); the other entries describe one illustrative setting, given exactly in Appendix B. ACT-R appears without retrieval as use.

11 Checking the formalization

Lean guarantees that the theorems follow from the definitions. It does not guarantee that the definitions say what the prose model says. We tested that link with an independent implementation that could not simply copy the first [cf. N-version programming, 13].

Method. The model was written as a prose specification, with no Lean names, theorems or predictions (`MultiDigit/sim/SPEC.md`). Points the prose leaves open were listed for the implementer to settle and document. Before any simulator existed, a list of checks was committed (`PREREG.md`). Each check named a theorem, the observable statistic, the hypothesis as read from the simulator's logs, and a tolerance: zero violations for exact relations, and frequencies within 4σ for probabilities. A check whose hypothesis never arose was to be reported as vacuous, not as passed. A fresh AI agent, given only the specification and working in an empty directory, wrote the simulator. An audit of its tool calls showed that it read only files it had written. Lean exported exact trial distributions for four instances, and a harness ran the checks on 30 random fixed instances and 20 random learning instances.

Agreement. Every check whose hypothesis arose passed, with no failures. 243 outcome frequencies matched Lean's exact distributions. The predictions about routes, confident errors, carrying, column errors, zeros and thresholds, and learning (non-interference, discrimination, then column work) passed on hundreds to hundreds of thousands of tests each. The simulation predates the problem solver. Because the solver is proved equal to the column procedure the simulation implements (`Solver.bridge`), its results carry over.

What the simulation found that the proofs did not. Three findings remain open in the current library.

1. **Hypotheses unrealized in the tested conditions.** Five predictions (near misses within 200 or 20, round problems, and column facts) were vacuous in the simulation: their hypotheses never held in any of the nets EPAM learned across the simulation's random instances. Each

Kind	Results	What they rest on
Library facts: properties of mechanisms, needed for reuse	the solver equals the column procedure (the bridge); the solver and the integrated trial are probability distributions; merged evaluation is the same distribution	the definitions, and nonnegative strengths with positive totals
Compositional deductions: hold for every encoding, net, strength set and DOA setting	whole-problem errors are independent of the solver; after certain recognition as q (as with any net), the column route factors as exhaustion times the solver, and backup use is $(1 - R_q)^n$	the composed mechanisms alone
Conditional: hold given stated conditions	confident errors; two sources of error; a solver error as a single-digit error in place (other columns exact); exactness of the solver (exact facts: sufficient, not necessary); consequences of representational properties (zeros, magnitude, column facts); noninterference with one memory	recognition conditions (derivable for EPAM nets, verified for the learned net); sharp associations (an unexplained empirical premise); exact columns; properties of the encoding; learned digit pairs
Class-level: hold for every mechanism satisfying stated laws	confident errors for any memory with determinacy, certainty and stability; which memory families have the laws (EPAM at every setting; ACT-R at none)	the laws themselves, which are not claimed to describe people
At chosen settings: exact values	which problems are confused; route, solver and error probabilities; the EPAM/ACT-R comparisons; counterexamples at a setting (Appendix B)	an encoding, a learned net, associations, criterion and search length chosen for illustration

Table 2: The results, by kind. Lean names are in Appendix A.

needs a path that tests *every* feature of a digit or column, and EPAM grows one test per confusion, so paths in these histories stayed short. This does not show that the hypotheses are unreachable. No impossibility theorem exists, and a long enough history of confusions on one digit could plausibly produce such paths. It shows that the theorems say little about the nets these learning histories produced. Proof alone did not reveal this; counting how often each hypothesis arose did. The distinction is between logical correctness and the model states that learning actually produces.

2. **A gap at zero strength.** With associations of total strength 0, sampling divides by zero, and a trial has total probability 0. The theorems avoid the case by assuming positive totals. The simulator, reading the prose, treats it as exhausted retrieval, which is the natural reading.

3. **An unstated assumption decides a predicted learning sequence.** The combined model suggests a sequence: a confident error, then, after discrimination, slow column work, then fluent correct retrieval. Whether it occurs depends on the initial associations of a newly discriminated problem, which the prose left open. With none, 40 of 48 discriminated problems followed the sequence. With only the correct answer, the column phase was skipped. With a copy of the neighbour's associations, the error was stated, strengthened and stated again: 7 of 44 problems were trapped, and 35 were still mixed at the end. The blind implementer flagged this dependence from the specification alone.

The simulation does not prove the formalization faithful to the intended theory. It is indepen-

dent evidence that the formal model behaves as its prose describes, and it found where the formal model says too little. Three standards are involved: logical correctness (Lean), behavior in the states learning produces (simulation), and empirical adequacy (data about people), which neither addresses.

12 Discussion

12.1 What this example shows

The companion paper places CogLean among earlier formal work on cognitive theory and relative to cognitive architectures [4]; we do not repeat that here. What this example adds is a demonstration that the combination it describes works on a task of some size: mechanisms formalized independently in a common environment; general theorems quantified over arbitrary representations and parameter settings; results transferred through instantiation and proved bridges; compositional deductions from independently established results; theory-neutral phenomenon predicates; results for classes of memory mechanisms satisfying common laws, with proofs of which families satisfy them; and concrete settings at which model families provably differ. It does not show the weakest assumptions sufficient for confident errors: the laws of Section 7 are shown sufficient, not necessary. Nor does it establish general equivalence or divergence among architectures, rather than counterexamples at settings.

12.2 What the approach bought

The examples above show four kinds of gain, of different weight. Some results are *library facts*, such as the bridge and the well-formedness of the composed distributions: unglamorous, but they are what lets theorems transfer. Some are *compositional deductions*, true at every setting: whole-problem errors do not depend on the solver. Some are *conditional*, and the conditions are where the science is: confident errors need confusion and sharp associations, and the proofs say exactly which premise a failed test would implicate. And some are *clarifications*: the ambiguity in the DOA chapter's claim about peaking, the theorem hypotheses that learning did not realize, and the unstated assumption behind the predicted learning sequence. None of these is yet an empirical result about people.

Proof and simulation were complementary here, in a way the slogan "simulation explores, proof settles" understates. The simulation checked what the proofs are *about*: whether their hypotheses arise, and whether the definitions say what the prose says.

12.3 Limitations

- The associative strengths in the worked example are chosen, not derived from experience. Siegler and Shrager's priors came from a counting process. Grounding these in learning histories is open.
- The strongest results depend on sharp associations and on particular recognition confusions. Confusions are derived for learned nets; sharpness is not.
- Single-digit DOA assumes that a representation with the right number of objects is available for elaboration and counting. As the column process of a multi-digit adder, its fallback to fingers and counting is implausible: by then single-digit facts are retrieved. The fallback should be a parameter of the column process; the general theorems do not depend on it.

- ACT-R's memory is represented by its retrieval rule, without retrieval as use: memory changes when an item is studied, not when it is retrieved during a trial. Every ACT-R value here, including Tables 3 and 4 and the counterexample to the pattern, holds for that version. Full ACT-R, in which each retrieval strengthens what it retrieves, would give different values, and possibly different theorems.
- Carrying is error-free in the full procedure, and columns are added independently.
- Scalability is a serious question. CogLean uses exact rational distributions over finite outcome spaces. Continuous representations, large state spaces and neural models will need different techniques, and exact evaluation already required a merged evaluator here.
- Three findings of the simulation remain open: hypotheses unrealized in the tested learning histories, the zero-strength gap, and the initial associations of a newly discriminated problem.
- CogLib's distributions are rational-weighted lists; the type does not enforce nonnegative weights. Most library theorems are algebraic identities that hold regardless, but reading them as probabilities needs nonnegativity. It is proved here for the integrated model, given nonnegative strengths and rates, but not yet across the whole library.

12.4 Future work

Natural next steps:

- derive associative distributions from learning histories, so that sharpness becomes a theorem;
- add feedback-driven learning across whole trials, so that the predicted learning sequence can be proved;
- restate the vacuous predictions in terms of the features a net actually tests, or prove when learning can realize their hypotheses;
- generalize the retriever interface, so that more memories can be swapped in, and so that retrieval can change memory (ACT-R's retrieval as use), with the memory's state carried through the solver;
- compose theories in other domains.

The library already contains early work on the last of these: conceptual combination, semantic memory and view application, on a shared semantic-network representation.

The companion paper describes directions for the library as a whole, including proof-guided model construction [4].

12.5 Conclusion

The question was whether theories represented in a common formal library can be composed to yield meaningful consequences that would otherwise require building a new model from scratch. In one case the answer is yes. EPAM's recognition, DOA's retrieval and strategy choice, and a decision-cycle problem solver, each formalized on its own, compose into a model of three-digit addition with no answer supplied, and its consequences are proved largely by reusing the components' theorems. The more important results are not the model's numbers but the statements above them: a phenomenon defined apart from any mechanism, a theorem relating it to recognition

and retrieval for any task and representation, and a theorem establishing it for every memory that satisfies three laws. The multi-digit model is a preliminary demonstration of method, not a proposed theory of arithmetic. The aspiration is a cumulative, machine-checked mathematical science of cognitive theories; one example does not establish that, but it shows that the approach can produce results of the right kind.

What accumulates is explanation. In this one example, a phenomenon has more than one explanation (whole-route and column-route errors; confident errors by composition and by laws), a mechanism takes part in explaining several phenomena, and the explanations differ in observations that could tell them apart, such as error signatures and backup use on confused problems. Because the explanations are proofs about the program that runs, they stay aligned with the model as both change.

Code. All definitions and theorems are in the CogLean repository [1], `https://github.com/jeffshrager/CogLean`. Every theorem is fully proved, with no `sorry`, and depends only on Lean's standard axioms.

References

[1] Jeff Shrager. CogLean: A library of cognitive theory in Lean 4. `https://github.com/jeffshrager/CogLean`, 2026.

[2] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

[3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[4] Jeff Shrager. CogLean: Towards a cumulative, machine-checked theory of cognition. Draft for comment, `papers/2026-coglean-intro/` in the CogLean repository, 2026.

[5] Allen Newell. *Unified Theories of Cognition*. Harvard University Press, Cambridge, MA, 1990.

[6] John R. Anderson. *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, New York, 2007.

[7] John E. Laird. *The Soar Cognitive Architecture*. MIT Press, Cambridge, MA, 2012.

[8] Edward A. Feigenbaum. An information processing theory of verbal learning. RAND Paper P-1817, RAND Corporation, Santa Monica, CA, 1959.

[9] Edward A. Feigenbaum and Herbert A. Simon. A theory of the serial position effect. *British Journal of Psychology*, 53(3):307–320, 1962. doi: 10.1111/j.2044-8295.1962.tb00836.x.

[10] Robert S. Siegler and Jeff Shrager. Strategy choices in addition and subtraction: How do children know what to do? In Catherine Sophian, editor, *Origins of Cognitive Skills*, pages 229–293. Erlbaum, Hillsdale, NJ, 1984.

[11] Allen Newell and Herbert A. Simon. *Human Problem Solving*. Prentice-Hall, Englewood Cliffs, NJ, 1972.

[12] John Seely Brown and Kurt VanLehn. Repair theory: A generative theory of bugs in procedural skills. *Cognitive Science*, 4(4):379–426, 1980. doi: 10.1207/s15516709cog0404_3.

[13] Algirdas Avižienis. The N-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering*, SE-11(12):1491–1501, 1985. doi: 10.1109/TSE.1985.231893.

A Index of results

B The illustrative setting and exact values

The setting. The encoding codes each digit of each addend in four bits. The net is learned by EPAM from $100 + 100$ and $100 + 200$, scanning hundreds digits first, so it tests only the bit where those two problems differ. The whole-problem associations give strength .90 to each familiar problem's correct sum and .01 to other answers, with criterion .50 and search length 1. Every digit pair has the same single-digit associations: 4/5 on its sum, 3/25 on the next number in the counting sequence, and 1/100 on each other answer from 0 to 19. The single-digit criterion is 1/10, the search length 1, and finger elaboration, skipping and double-counting are each 1/20. ACT-R's parameters are a mismatch penalty $\mu = 1/2$ over 24 features and a failure weight $\theta = 1/10$, with decay 1 and temperature 1 so that probabilities stay rational (`CogLib/Memory/Activation.lean`).

EPAM. $150+100$ is answered 200 by whole-problem retrieval with probability 45/46 and reaches the solver with probability 1/46. The solver alone answers 250 with probability $\approx .656$. Its main errors are 260, 350 and 251, each $\approx .1$: a tens error, a hundreds error (the counting-sequence neighbour of $1 + 1$) and a units error. So $150 + 100$ is answered correctly with probability $\approx .0143$ (`setting_routes`, `setting_solver_x`, `setting_overall_x`), and the familiar $100 + 100$ with probability $\approx .993$. The model has `ConfidentErrors` with bound 1/46 (`setting_confident_errors_bound`).

ACT-R. With the confusion graded, $150 + 100$ can retrieve either familiar problem, or fail. It reaches the solver far more often and is answered correctly about a quarter of the time. ACT-R also predicts interference: studying $100 + 300$ roughly doubles the retrieval errors on the practiced $100+100$ (`act_interference`), and raises its overall errors from about a third to about three fifths. EPAM's are unchanged (`MD.learning_preserves_trial`).

One memory. The memory first studies $0+0$, $5+0$ and $1+1$, then the two problems. EPAM's net recognizes all five as themselves, and every value is the integrated model's (`setting_epam_values`). ACT-R's memory confuses digit pairs: $0 + 0$ calls up $5 + 0$ with probability 15/197. Two things change at once for ACT-R, the memory holds three more items and the solver consults it, and a control column separates them. With the extra items but the separate table, correct answers to $150+100$ rise slightly (.267, against .238), since more problems reach an accurate solver. Using the memory inside the solver lowers them: the solver's accuracy when invoked falls from .656 to .385, correct answers to .157, and errors on $100 + 100$ rise to .386. Part of ACT-R's loss comes from the fallback to finger counting, which a child doing three-digit addition would not use.

These numbers are exact consequences of the setting. They are *not* fitted to children's data and are not predictions about children. (Computing them needed merged evaluators, proved to be the same distributions as the model, `SC.same_trialMerged`: written out, counting k objects lists 3^k outcomes.)

Claim	Lean name (file)
Discrimination learning preserves prior recognition	`learn_preserves` (`CogLib/Process/DiscriminationLearning`)
A confusion agrees on every tested feature	`confusion_agrees_on_path` (`CogLib/Memory/DiscriminationNet`)
Learned nets recognize each image as itself	`learnAll_clean` (`CogLib/Process/DiscriminationLearning`)
DOA retrieval use, error share, backup independence	`SC.retrieval_prob`, `retrieval_error_share`, `retrieval_independent_of_backup`
Counting errors independent of associations	`SC.counting_error_share`
Peaking never raises retrieval errors; but can raise overt strategy use	`SC.peaking_lowers_retrieval_errors`; `SC.peaking_can_raise_overt_use`
Learning never raises backup use above the criterion; can below it, or with renormalization	`reach_boost`, `exhausted_boost`; `SC.correct_strengthening_never_raises_backup`, `SC.dominant_strengthening_never_raises_backup`; `SC.strengthening_below_criterion_can_raise_backup`, `SC.normalized_strengthening_can_raise_backup`
The phenomena: confusion-error pattern; confident errors with a bound	`ConfusionErrorPattern`, `ConfidentErrors` (`CogLib/Phenomena/ConfusionErrors`)
Confident errors, any task; with the rate as bound	`RR.confident_errors`, `RR.confident_errors_rate` (`CogLib/Process/RecognizeRetrieve`)
Confident errors from laws of memory	`RR.confident_errors_of_laws`
Solver equals the column procedure	`Solver.bridge` (`MultiDigit/Solver`)
A column error is a DOA error in place	`MD.column_error_is_doa_error` (`MultiDigit/Columns`)
Integrated: solver is the column backup; mass 1; nonnegative weights	`solverBackup_eq_columns`, `mass_solverBackup`, `nonneg_solverBackup`, `nonneg_itrial` (`MultiDigit/Integrated`)
Integrated: rate of solver use, and its bound	`integrated_backup_rate`, `integrated_backup_le`
Integrated: whole-problem errors independent of solver	`whole_independent_of_solver`
Integrated: column route factored	`columnsSuch_prob`
One memory: column fact is DOA under self-recognition; reduction to the integrated model	`memFact_self`, `strial_reduces`, `strial_net_reduces` (`MultiDigit/SharedMemory`)
One memory: confident errors on digit pairs; practice undisturbed given learned pairs	`column_confident_errors`, `shared_learning_preserves_trial`
One memory: values at the setting; ACT-R control	`setting_epam_values`, `setting_epam_solver`, `setting_00_not_5`, `setting_act_pairs`, `setting_act_values`; `setting_act_control`
Integrated: confident errors; with a bound; two sources	`integrated_confident_errors`, `integrated_confident_errors_le`, `errors_two_sources`
Integrated: solver error is a DOA error (other columns exact); exact facts sufficient, not necessary	`solver_error_is_doa_error`, `solver_exact_of_exact`, `compensating_errors`
Integrated: values at the setting (EPAM; ACT-R)	`setting_routes`, `setting_solver_x`, `setting_overall_x`, `setting_overall_q1`, `setting_practiced_errors`, `setting_valid`, `setting_confident_errors_bound`; `setting_actr_x`, `setting_actr_not_pattern`
ACT-R: graded confusion; interference	`act_values`, `act_interference` (`MultiDigit/ExampleACTR`)
EPAM: practice never disturbed	`MD.learning_preserves_trial`
Memory laws: EPAM has, ACT-R never has	`Matrix.Discrimination.*`, `Matrix.Activation.never_*` (`CogMatrix`)
Merged evaluation is the same distribution	`SC.same_trialMerged`, `same_fastBackup`

A. Separate column-fact table. Memory holds $100 + 100$, $100 + 200$; columns use DOA's own table

	EPAM net	ACT-R memory
$150 + 100$ retrieves $100 + 100$	certainly	with probability $10/23$
$150 + 100$ answered 200 by whole-problem retrieval	$45/46$	$225/529 \approx .43$
$150 + 100$ answered 300 (taken for $100 + 200$)	0	$225/1058 \approx .21$
$150 + 100$: solver invoked	$1/46 \approx .022$	$383/1058 \approx .36$
$150 + 100$: solver correct, *given* invoked	$\approx .656$	$\approx .656$
$150 + 100$ answered correctly (250) overall, all via the solver	$\approx .014$	$\approx .238$
practiced $100+100$: errors by whole-problem retrieval	0	$225/782 \approx .29$
practiced $100 + 100$: errors overall	$\approx .0073$	$\approx .334$
after also studying $100+300$: whole-problem retrieval errors	0	$3375/5842 \approx .58$
after also studying $100 + 300$: overall	$\approx .0073$ (unchanged)	$\approx .616$
for every setting, with this solver	confident-error theorem applies; practice never disturbed	its hypotheses never hold; interference possible

Table 3: The integrated model with two memories, at one illustrative setting: the binary-coded encoding, both memories having studied $100 + 100$ and then $100 + 200$, the same associations, criterion $1/2$, search length 1, and the same solver, whose single-digit facts come from DOA's separate association table. ACT-R's parameters are a mismatch penalty $\mu = 1/2$ over 24 features and a failure weight $\theta = 1/10$. All values are exact and proved in Lean. "Correct overall" is the joint probability that the solver is invoked and is correct: the product of the two rows above it (only the solver ever gives 250). Rows about whole-problem retrieval do not depend on the backup (`whole_independent_of_solver`), so they are the same in `ExampleACTR.lean`, which uses an exact backup. Rows about answers overall use the integrated model (`setting_routes`, `setting_solver_x`, `setting_actr_x`, `setting_practiced_errors`). EPAM's overall errors on $100 + 100$ are the solver's own; after studying $100 + 300$ they are unchanged, by `MD.learning_preserves_trial`, which holds for every backup that learning leaves fixed, as this solver's table is. The last row states what holds at every setting for this configuration.

B. One memory. Memory holds $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$			
	EPAM net one memory	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 by the column process	0	0	120/2167
$150+100$ answered 200 by whole-problem retrieval	45/46	$\approx .396$	$\approx .396$
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
$150+100$: solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
$150+100$ answered correctly overall	$\approx .014$	$\approx .267$	$\approx .157$
practiced $100+100$: errors overall	$\approx .0073$	$\approx .334$	$\approx .386$
For every setting, EPAM: practice undisturbed, given that the problem and its digit pairs were learned.			

Table 4: The model with one memory for whole problems and column facts (§6.4). The memory first studies the three digit pairs that $150 + 100$ and $100 + 100$ need, then the two problems. The middle column is a control: ACT-R's memory holds the same five items, but the solver uses DOA's separate table, so the memory's contents are as in the last column and only the solver's use of the memory differs. Rows as in Table 3; "correct overall" is again the product of the two rows above it. EPAM's values are those of Table 3, by the reduction theorem (`setting_epam_reduces`, `setting_epam_values`, `setting_epam_solver`, `setting_00_not_5`); ACT-R's are `setting_act_control`, `setting_act_pairs` and `setting_act_values`. ACT-R appears without retrieval as use. The last row is `shared_learning_preserves_trial`; without its condition on the digit pairs, noninterference is not proved for the shared model.